

4.05-hpi-unos-full-dirac-gpu

September 2, 2024

```
[1]: %load_ext autoreload
      %autoreload 2

      import numpy as np
      import pandas as pd
      import torch

      from prism.config import PROCESSED_DATA_DIR, MODELS_DIR
      from prism.legacy import normalise, modelStats
      from prism.maskedmlp import train_mlp_batched, mlpmask_pytorch
      from prism.save_models import save_mlp
      from prism.partial_responses import partial_responses
      from prism.nomogram import nomogram
      from prism.lasso import lasso
      from prism.device_tools import get_device, print_tensor_info
```

```
2024-08-23 16:48:15.183 | INFO      |
prism.config:<module>:11 - PROJ_ROOT path is:
C:\Users\localuser\PRiSM\prism_github
```

```
[2]: %reload_ext autoreload
```

0.1 Set computation device and parameters

```
[3]: # By default, assign device according to CUDA > MPS > CPU
      device = get_device()
      print(f"Using device: {device}")
```

Using device: cuda

```
[4]: # Optionally, overwrite the default device to CPU
      # device = 'cpu'
      # print(f"Using device: {device}")
```

```
[5]: # Parameters
      method = 'dirac'
      SAVE_MODELS = False
```

```
seed = 257
np.random.seed(seed)
torch.manual_seed(seed)
```

[5]: <torch._C.Generator at 0x1bf7fe8afb0>

0.2 Load and preprocess data

```
[6]: data_train = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_train.
    ↪csv'))
data_test = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_test.
    ↪csv'))
data_val = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_val.csv'))

data_train_test = pd.concat([data_train,data_test]) # used for axis annotation,
    ↪in some plotting functions

# drop id column
data_train.drop('trr_id_code',axis=1,inplace=True)
data_test.drop('trr_id_code',axis=1,inplace=True)
data_val.drop('trr_id_code',axis=1,inplace=True)

target_col = 'oneyearmort'

x_train0 = data_train.drop(target_col,axis=1)
y_train = data_train[target_col]

x_test0 = data_test.drop(target_col,axis=1)
y_test = data_test[target_col]

x_val0 = data_val.drop(target_col,axis=1)
y_val = data_val[target_col]

[x_train,x_test] = normalise(x_train0,x_test0)
x_val = normalise(x_val0)

x_train_tensor = torch.tensor(x_train.values, dtype=torch.float32,
    ↪device=device)
x_test_tensor = torch.tensor(x_test.values, dtype=torch.float32, device=device)
x_val_tensor = torch.tensor(x_val.values, dtype=torch.float32, device=device)

x_train0_median = x_train0.median().values
x_train0_std = x_train0.std().values
```

```
[7]: feature_names = [
    'don age',
    'don isch time min',
```

```

'rec age yr',
'rec creat',
'rec infect 2wk',
'rec vent',
'rec sex',
'tx year',
'ICM',
'NICM',
'prior tx'
]

```

0.3 Train MLP

```

[8]: mlp_params = {
    'n_hidden': 10,
    'weight_decay': 1e-5,
    'lr': 0.001,
    'patience': 50,
    'tolerance': 0.0001,
    'batch_size': 1024,
    'device': device,
    'seed': seed
}

mlp = train_mlp_batched(x_train, y_train, x_test, y_test, **mlp_params)

```

```

Epoch 0: Train loss 0.6690, Val loss 0.6166
Epoch 1: Train loss 0.5749, Val loss 0.5303
Epoch 2: Train loss 0.5062, Val loss 0.4680
Epoch 3: Train loss 0.4573, Val loss 0.4224
Epoch 4: Train loss 0.4240, Val loss 0.3906
Epoch 5: Train loss 0.4014, Val loss 0.3693
Epoch 6: Train loss 0.3869, Val loss 0.3563
Epoch 7: Train loss 0.3782, Val loss 0.3485
Epoch 8: Train loss 0.3726, Val loss 0.3440
Epoch 9: Train loss 0.3694, Val loss 0.3416
Epoch 10: Train loss 0.3667, Val loss 0.3402
Epoch 11: Train loss 0.3653, Val loss 0.3395
Epoch 12: Train loss 0.3644, Val loss 0.3393
Epoch 13: Train loss 0.3640, Val loss 0.3392
Epoch 14: Train loss 0.3631, Val loss 0.3391
Epoch 15: Train loss 0.3621, Val loss 0.3391
Epoch 16: Train loss 0.3624, Val loss 0.3392
Epoch 17: Train loss 0.3617, Val loss 0.3392
Epoch 18: Train loss 0.3613, Val loss 0.3393
Epoch 19: Train loss 0.3611, Val loss 0.3392
Epoch 20: Train loss 0.3611, Val loss 0.3393
Epoch 21: Train loss 0.3608, Val loss 0.3394

```

Epoch 22: Train loss 0.3601, Val loss 0.3392
Epoch 23: Train loss 0.3604, Val loss 0.3392
Epoch 24: Train loss 0.3604, Val loss 0.3392
Epoch 25: Train loss 0.3601, Val loss 0.3390
Epoch 26: Train loss 0.3598, Val loss 0.3390
Epoch 27: Train loss 0.3593, Val loss 0.3389
Epoch 28: Train loss 0.3598, Val loss 0.3391
Epoch 29: Train loss 0.3586, Val loss 0.3388
Epoch 30: Train loss 0.3588, Val loss 0.3389
Epoch 31: Train loss 0.3587, Val loss 0.3388
Epoch 32: Train loss 0.3584, Val loss 0.3389
Epoch 33: Train loss 0.3585, Val loss 0.3388
Epoch 34: Train loss 0.3584, Val loss 0.3388
Epoch 35: Train loss 0.3588, Val loss 0.3387
Epoch 36: Train loss 0.3583, Val loss 0.3386
Epoch 37: Train loss 0.3581, Val loss 0.3386
Epoch 38: Train loss 0.3583, Val loss 0.3385
Epoch 39: Train loss 0.3576, Val loss 0.3384
Epoch 40: Train loss 0.3577, Val loss 0.3384
Epoch 41: Train loss 0.3574, Val loss 0.3385
Epoch 42: Train loss 0.3572, Val loss 0.3384
Epoch 43: Train loss 0.3567, Val loss 0.3383
Epoch 44: Train loss 0.3573, Val loss 0.3385
Epoch 45: Train loss 0.3568, Val loss 0.3383
Epoch 46: Train loss 0.3569, Val loss 0.3384
Epoch 47: Train loss 0.3573, Val loss 0.3383
Epoch 48: Train loss 0.3570, Val loss 0.3380
Epoch 49: Train loss 0.3573, Val loss 0.3382
Epoch 50: Train loss 0.3567, Val loss 0.3381
Epoch 51: Train loss 0.3571, Val loss 0.3381
Epoch 52: Train loss 0.3568, Val loss 0.3379
Epoch 53: Train loss 0.3562, Val loss 0.3378
Epoch 54: Train loss 0.3567, Val loss 0.3377
Epoch 55: Train loss 0.3564, Val loss 0.3379
Epoch 56: Train loss 0.3566, Val loss 0.3380
Epoch 57: Train loss 0.3558, Val loss 0.3378
Epoch 58: Train loss 0.3564, Val loss 0.3379
Epoch 59: Train loss 0.3559, Val loss 0.3376
Epoch 60: Train loss 0.3550, Val loss 0.3377
Epoch 61: Train loss 0.3555, Val loss 0.3378
Epoch 62: Train loss 0.3561, Val loss 0.3378
Epoch 63: Train loss 0.3547, Val loss 0.3377
Epoch 64: Train loss 0.3551, Val loss 0.3377
Epoch 65: Train loss 0.3556, Val loss 0.3378
Epoch 66: Train loss 0.3553, Val loss 0.3375
Epoch 67: Train loss 0.3555, Val loss 0.3377
Epoch 68: Train loss 0.3552, Val loss 0.3376
Epoch 69: Train loss 0.3547, Val loss 0.3377

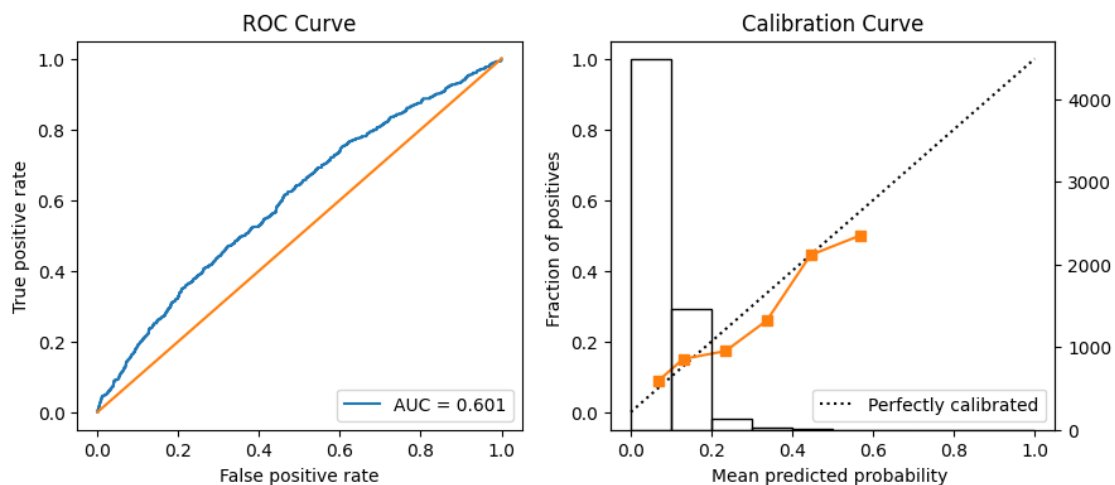
Epoch 70: Train loss 0.3555, Val loss 0.3376
Epoch 71: Train loss 0.3549, Val loss 0.3378
Epoch 72: Train loss 0.3546, Val loss 0.3376
Epoch 73: Train loss 0.3546, Val loss 0.3376
Epoch 74: Train loss 0.3550, Val loss 0.3375
Epoch 75: Train loss 0.3541, Val loss 0.3375
Epoch 76: Train loss 0.3547, Val loss 0.3378
Epoch 77: Train loss 0.3543, Val loss 0.3376
Epoch 78: Train loss 0.3545, Val loss 0.3375
Epoch 79: Train loss 0.3542, Val loss 0.3374
Epoch 80: Train loss 0.3552, Val loss 0.3375
Epoch 81: Train loss 0.3544, Val loss 0.3374
Epoch 82: Train loss 0.3538, Val loss 0.3376
Epoch 83: Train loss 0.3546, Val loss 0.3375
Epoch 84: Train loss 0.3540, Val loss 0.3374
Epoch 85: Train loss 0.3552, Val loss 0.3374
Epoch 86: Train loss 0.3548, Val loss 0.3376
Epoch 87: Train loss 0.3542, Val loss 0.3373
Epoch 88: Train loss 0.3540, Val loss 0.3376
Epoch 89: Train loss 0.3542, Val loss 0.3375
Epoch 90: Train loss 0.3542, Val loss 0.3376
Epoch 91: Train loss 0.3540, Val loss 0.3375
Epoch 92: Train loss 0.3543, Val loss 0.3375
Epoch 93: Train loss 0.3537, Val loss 0.3374
Epoch 94: Train loss 0.3539, Val loss 0.3377
Epoch 95: Train loss 0.3543, Val loss 0.3375
Epoch 96: Train loss 0.3540, Val loss 0.3374
Epoch 97: Train loss 0.3537, Val loss 0.3377
Epoch 98: Train loss 0.3535, Val loss 0.3374
Epoch 99: Train loss 0.3537, Val loss 0.3374
Epoch 100: Train loss 0.3534, Val loss 0.3375
Epoch 101: Train loss 0.3542, Val loss 0.3376
Epoch 102: Train loss 0.3539, Val loss 0.3376
Epoch 103: Train loss 0.3540, Val loss 0.3377
Epoch 104: Train loss 0.3532, Val loss 0.3373
Epoch 105: Train loss 0.3537, Val loss 0.3378
Epoch 106: Train loss 0.3537, Val loss 0.3375
Epoch 107: Train loss 0.3535, Val loss 0.3374
Epoch 108: Train loss 0.3535, Val loss 0.3377
Epoch 109: Train loss 0.3533, Val loss 0.3377
Epoch 110: Train loss 0.3535, Val loss 0.3377
Epoch 111: Train loss 0.3536, Val loss 0.3376
Epoch 112: Train loss 0.3539, Val loss 0.3379
Epoch 113: Train loss 0.3533, Val loss 0.3376
Epoch 114: Train loss 0.3539, Val loss 0.3377
Epoch 115: Train loss 0.3536, Val loss 0.3379
Epoch 116: Train loss 0.3543, Val loss 0.3379
Epoch 117: Train loss 0.3534, Val loss 0.3376

Epoch 118: Train loss 0.3534, Val loss 0.3376
Epoch 119: Train loss 0.3545, Val loss 0.3378
Epoch 120: Train loss 0.3538, Val loss 0.3379
Epoch 121: Train loss 0.3538, Val loss 0.3374
Epoch 122: Train loss 0.3535, Val loss 0.3374
Epoch 123: Train loss 0.3536, Val loss 0.3377
Epoch 124: Train loss 0.3527, Val loss 0.3378
Epoch 125: Train loss 0.3535, Val loss 0.3378
Epoch 126: Train loss 0.3532, Val loss 0.3377
Epoch 127: Train loss 0.3538, Val loss 0.3380
Epoch 128: Train loss 0.3536, Val loss 0.3378
Epoch 129: Train loss 0.3537, Val loss 0.3378
Epoch 130: Train loss 0.3527, Val loss 0.3378
Epoch 131: Train loss 0.3532, Val loss 0.3376
Epoch 132: Train loss 0.3534, Val loss 0.3377
Epoch 133: Train loss 0.3533, Val loss 0.3378
Epoch 134: Train loss 0.3529, Val loss 0.3377
Epoch 135: Train loss 0.3537, Val loss 0.3379
Epoch 136: Train loss 0.3535, Val loss 0.3380
Stopping early at epoch 87

1 Evaluate MLP

```
[9]: y_test_blackbox = mlp.predict(x_test_tensor).cpu().numpy()
     y_val_blackbox = mlp.predict(x_val_tensor).cpu().numpy()

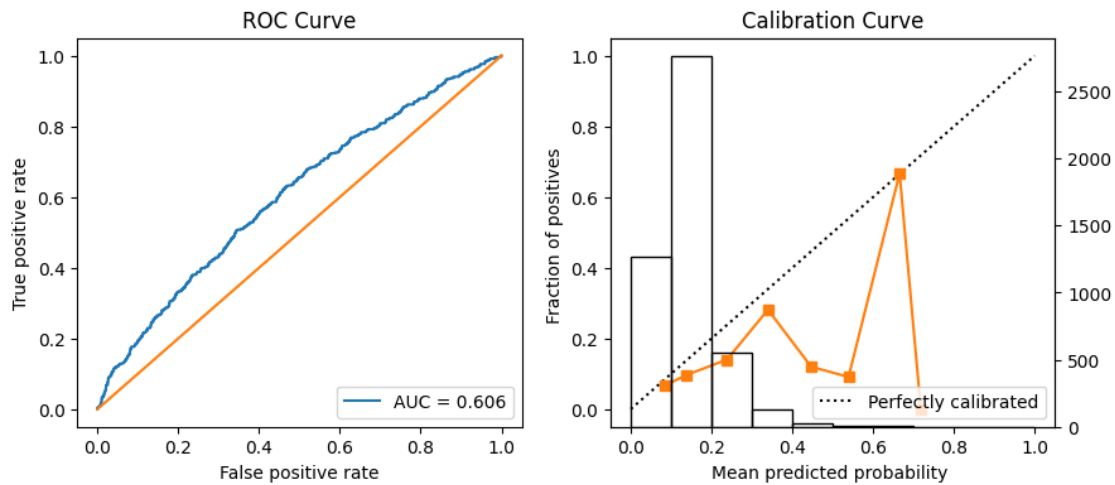
     mlp_metrics_test = modelStats(y_test_blackbox, y_test, y_train, ROC=True,
     ↪mdlCalibration=True, metricNames=True, auc_ci=True)
     modelStats(y_val_blackbox, y_val, y_train, ROC=True, mdlCalibration=True,
     ↪metricNames=True, auc_ci=True);
```



```

----- Metrics -----
prevalence: 0.123
sensitivity: 0.241
specificity: 0.864
accuracy: 0.797
ppv: 0.175
auc score: 0.601
auc lower ci: 0.577
auc upper ci: 0.623
-----

```



```

----- Metrics -----
prevalence: 0.123
sensitivity: 0.667
specificity: 0.489
accuracy: 0.507
ppv: 0.126
auc score: 0.606
auc lower ci: 0.582
auc upper ci: 0.631
-----

```

```

[10]: # Save MLP
      if SAVE_MODELS:
          save_mlp(mlp, mlp_params, mlp_metrics_test, MODELS_DIR)

```

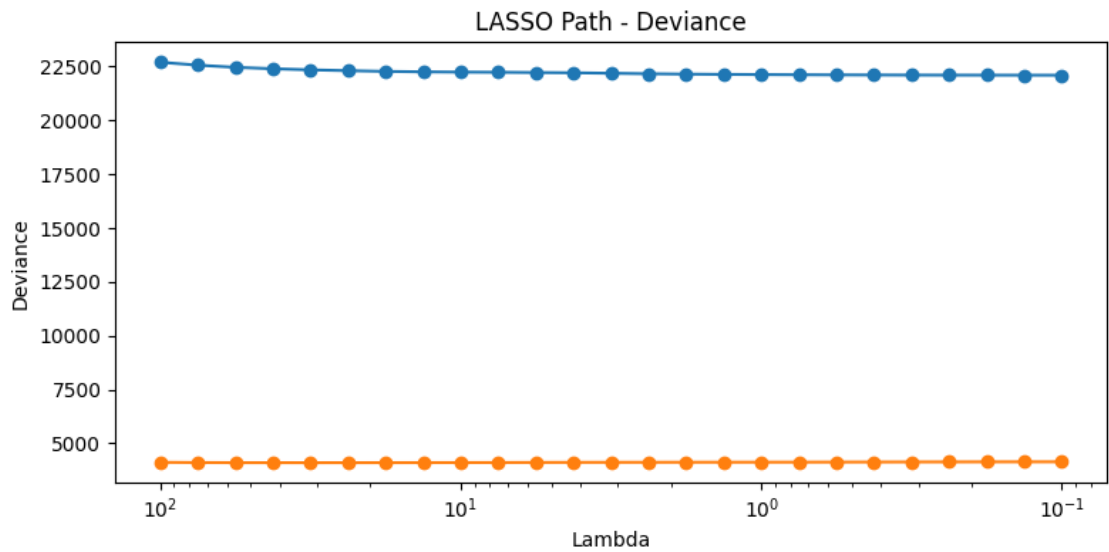
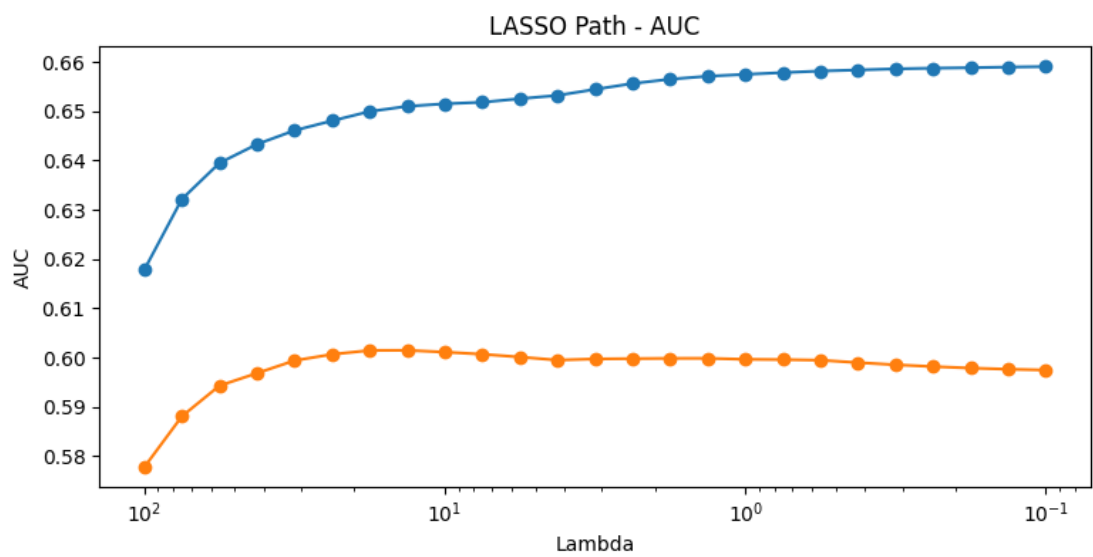
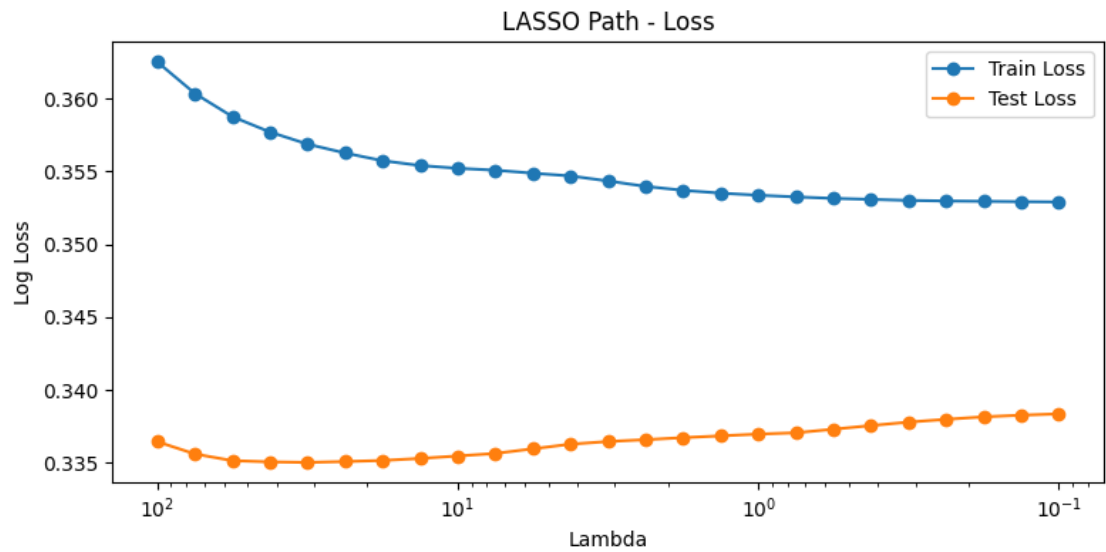
2 MLP LASSO

```
[11]: %reload_ext autoreload
# by default, max_workers (threads) = logical CPU cores - 1. Otherwise, set
↳ optional argument max_workers
# for single GPU (cuda): choose max_workers=1.
# for MPS (Apple Silicon): start with max_workers=1, then experiment with more
partial_responses_params = {
    'x_train' : x_train_tensor,
    'method' : method,
    'device' : device,
    'batch_size' : 512,
    'max_workers' : 1
}

partial_responses_train = partial_responses(x_train_tensor, mlp,
↳ **partial_responses_params)

partial_responses_test = partial_responses(x_test_tensor, mlp,
↳ **partial_responses_params)
```

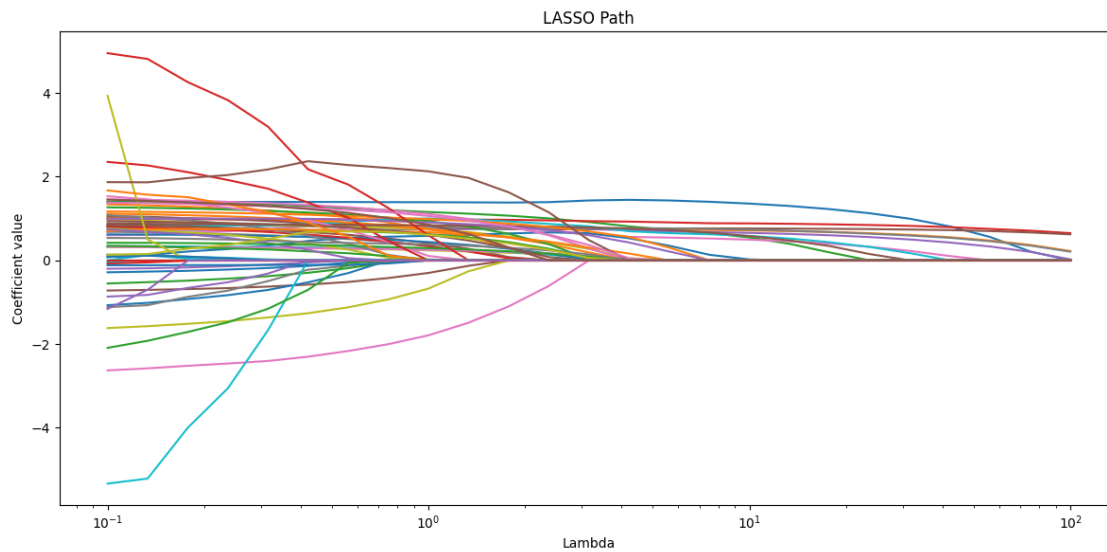
```
[12]: %reload_ext autoreload
lasso_results = lasso(
    partial_responses_train,
    partial_responses_test,
    y_train,
    y_test,
    feature_names=feature_names,
    nlambda=25,
    min_lambda=0.1,
    max_lambda=100,
    batch_size=2,
    seed=seed
)
```

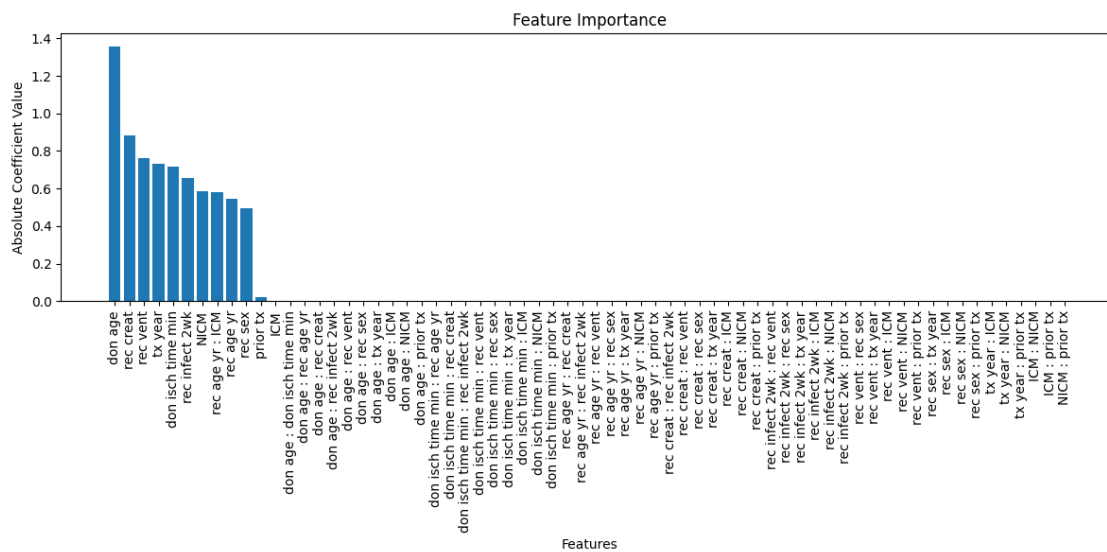
<IPython.core.display.HTML object>

All fits converged successfully.

```
[13]: lasso_results.plot_lambda_path()
```



```
[14]: lasso_results.select_lambda(8)
lasso_results.plot_feature_importance()
```



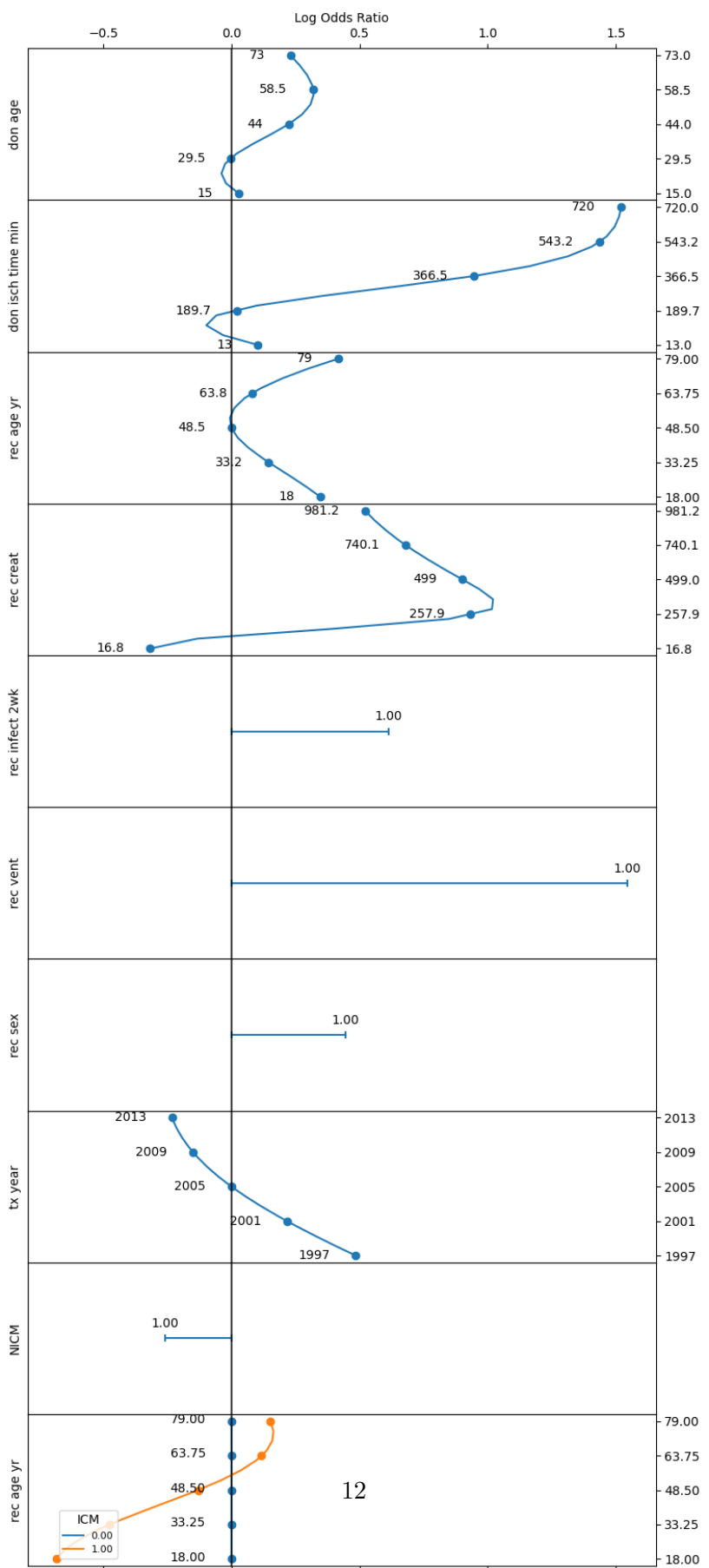
```
[15]: %reload_ext autoreload
# Note: The prPlots function is not yet available in the new implementation.
# We'll use the new nomogram function instead.
nomogram_results = nomogram(
    lasso_results,
    x_train_tensor,
    x_train0.median().values,
    x_train0.std().values,
    mlp,
    n_steps=15,
    sd_scale=2,
    method=method,
    device=device,
    categorical_threshold=15,
    subtract_univariate=True
);
```

C:\Users\localuser\PRiSM\prism_github\prism\partial_responses.py:271:

UserWarning: torch.searchsorted(): input value tensor is non-contiguous, this will lower the performance due to extra data copy when converting non-contiguous tensor to contiguous, please use contiguous input value tensor if possible. This message will only appear once per program. (Triggered internally at ../aten/src/ATen/native/BucketizationUtils.h:34.)

```
    bivariate_response -= (univariate_responses[i][torch.searchsorted(x_subset_i,
x_ij[:, 0])]) +
```

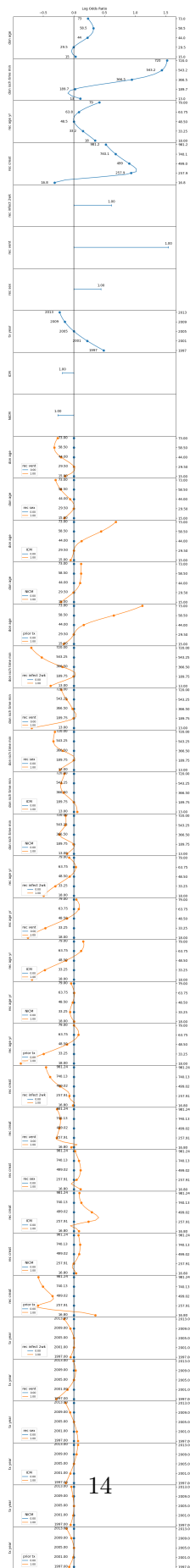
Nomogram of univariate and mixed bivariate partial responses



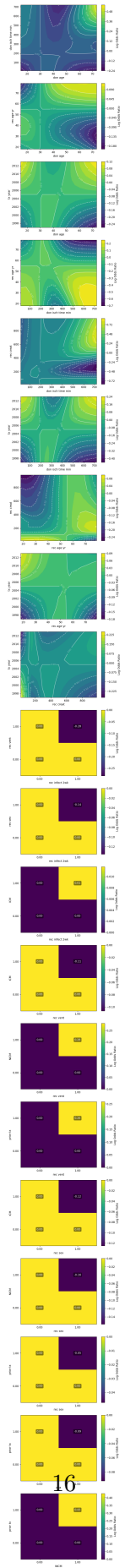
```
[16]: # test with more features
lasso_results.select_lambda(24)
```

```
[17]: %reload_ext autoreload
nomogram_results = nomogram(
    lasso_results,
    x_train_tensor,
    x_train0.median().values,
    x_train0.std().values,
    mlp,
    n_steps=15,
    sd_scale=2,
    method=method,
    device=device,
    categorical_threshold=15,
    subtract_univariate=True
)
```

Nonogram of univariate and mixed bivariate partial responses



Homograms of non-inbred kinship partial responses

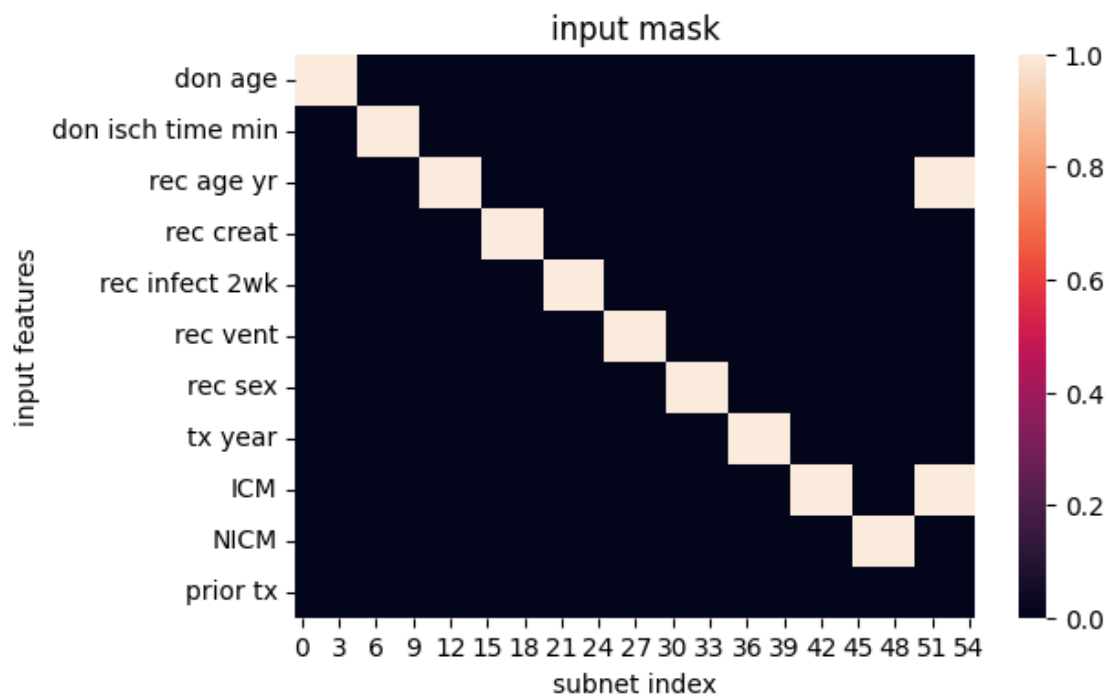



```
[18]: # return to previously selected lambda
lasso_results.select_lambda(8)
```

3 Train the Partial Response Network

```
[19]: mask, n_features = lasso_results.get_mask()
```

Selected features: ['don age', 'don isch time min', 'rec age yr', 'rec creat', 'rec infect 2wk', 'rec vent', 'rec sex', 'tx year', 'NICM', 'rec age yr : ICM']



```
[20]: prn_params = {
    'n_hidden': n_features,
    'mask': mask,
    'subnet_nodes': 5,
    'iter': 10000,
    'lr': 0.05,
    'weight_decay': 0.00001,
    'tolerance': 0.0001,
    'patience': 100,
    'device': device,
    'seed': seed
}
```

```
}
```

```
prn = mlpmask_pytorch(x_train, y_train, x_test, y_test, **prn_params)
```

```
Epoch 0, Training loss 0.7491950392723083, Validation loss 0.5058019161224365
Epoch 1, Training loss 0.5208706855773926, Validation loss 0.3845042884349823
Epoch 2, Training loss 0.4087001383304596, Validation loss 0.34338873624801636
Epoch 3, Training loss 0.3777749240398407, Validation loss 0.3566133379936218
Epoch 4, Training loss 0.3964877128601074, Validation loss 0.38072434067726135
Epoch 5, Training loss 0.42082345485687256, Validation loss 0.39131689071655273
Epoch 6, Training loss 0.4279251992702484, Validation loss 0.38494938611984253
Epoch 7, Training loss 0.41638854146003723, Validation loss 0.36732038855552673
Epoch 8, Training loss 0.3941255211830139, Validation loss 0.3479166626930237
Epoch 9, Training loss 0.3729674220085144, Validation loss 0.33749744296073914
Epoch 10, Training loss 0.3656228184700012, Validation loss 0.3415509760379791
Epoch 11, Training loss 0.37698015570640564, Validation loss 0.3493284583091736
Epoch 12, Training loss 0.39084920287132263, Validation loss 0.3470105230808258
Epoch 13, Training loss 0.3884521424770355, Validation loss 0.33864396810531616
Epoch 14, Training loss 0.37404555082321167, Validation loss 0.33528998494148254
Epoch 15, Training loss 0.3619023859500885, Validation loss 0.34104466438293457
Epoch 16, Training loss 0.35944950580596924, Validation loss 0.35184338688850403
Epoch 17, Training loss 0.3643118441104889, Validation loss 0.36123892664909363
Epoch 18, Training loss 0.37016409635543823, Validation loss 0.36496368050575256
Epoch 19, Training loss 0.3723222017288208, Validation loss 0.3621066212654114
Epoch 20, Training loss 0.3696381449699402, Validation loss 0.354486346244812
Epoch 21, Training loss 0.3639782965183258, Validation loss 0.3455081284046173
Epoch 22, Training loss 0.3587774336338043, Validation loss 0.3386857807636261
Epoch 23, Training loss 0.3570753335952759, Validation loss 0.33580371737480164
Epoch 24, Training loss 0.35936906933784485, Validation loss 0.3357852101325989
Epoch 25, Training loss 0.3628600537776947, Validation loss 0.3360856771469116
Epoch 26, Training loss 0.36388930678367615, Validation loss 0.33564335107803345
Epoch 27, Training loss 0.3615776002407074, Validation loss 0.335593044757843
Epoch 28, Training loss 0.35822033882141113, Validation loss 0.3372500538825989
Epoch 29, Training loss 0.3564668595790863, Validation loss 0.340413898229599
Epoch 30, Training loss 0.35701942443847656, Validation loss 0.3435826897621155
Epoch 31, Training loss 0.35869941115379333, Validation loss 0.3451804220676422
Epoch 32, Training loss 0.35986778140068054, Validation loss 0.3445168137550354
Epoch 33, Training loss 0.3596443831920624, Validation loss 0.3420269787311554
Epoch 34, Training loss 0.3582829236984253, Validation loss 0.3389122486114502
Epoch 35, Training loss 0.35680246353149414, Validation loss 0.33643555641174316
Epoch 36, Training loss 0.35620632767677307, Validation loss 0.3352040648460388
Epoch 37, Training loss 0.3567263185977936, Validation loss 0.3349232077598572
Epoch 38, Training loss 0.3576492667198181, Validation loss 0.3349165916442871
Epoch 39, Training loss 0.35799238085746765, Validation loss 0.3349255919456482
Epoch 40, Training loss 0.3574346601963043, Validation loss 0.3352886736392975
Epoch 41, Training loss 0.3565181791782379, Validation loss 0.33637866377830505
Epoch 42, Training loss 0.3559926152229309, Validation loss 0.33805719017982483
Epoch 43, Training loss 0.3561197519302368, Validation loss 0.3396962285041809
```

Epoch 44, Training loss 0.356579065322876, Validation loss 0.34061458706855774
Epoch 45, Training loss 0.35686591267585754, Validation loss 0.3404918611049652
Epoch 46, Training loss 0.3567216098308563, Validation loss 0.3394988179206848
Epoch 47, Training loss 0.35627955198287964, Validation loss 0.3381355404853821
Epoch 48, Training loss 0.355896532535553, Validation loss 0.3369191288948059
Epoch 49, Training loss 0.3558361828327179, Validation loss 0.3361293077468872
Epoch 50, Training loss 0.3560543954372406, Validation loss 0.33577653765678406
Epoch 51, Training loss 0.3562680184841156, Validation loss 0.3357725143432617
Epoch 52, Training loss 0.3562413156032562, Validation loss 0.33608707785606384
Epoch 53, Training loss 0.35600021481513977, Validation loss 0.33671995997428894
Epoch 54, Training loss 0.35576486587524414, Validation loss 0.3375648260116577
Epoch 55, Training loss 0.35571014881134033, Validation loss 0.3383626341819763
Epoch 56, Training loss 0.3558168411254883, Validation loss 0.33881381154060364
Epoch 57, Training loss 0.3559291660785675, Validation loss 0.33874815702438354
Epoch 58, Training loss 0.35591819882392883, Validation loss 0.33821582794189453
Epoch 59, Training loss 0.35578784346580505, Validation loss 0.33744341135025024
Epoch 60, Training loss 0.35564911365509033, Validation loss 0.3366982638835907
Epoch 61, Training loss 0.35560446977615356, Validation loss 0.3361610174179077
Epoch 62, Training loss 0.35565441846847534, Validation loss 0.335889607667923
Epoch 63, Training loss 0.3557095527648926, Validation loss 0.33587127923965454
Epoch 64, Training loss 0.35568973422050476, Validation loss 0.33608099818229675
Epoch 65, Training loss 0.35560184717178345, Validation loss 0.3364782929420471
Epoch 66, Training loss 0.35551896691322327, Validation loss 0.336969256401062
Epoch 67, Training loss 0.3554966449737549, Validation loss 0.3374041020870209
Epoch 68, Training loss 0.3555222153663635, Validation loss 0.3376361131668091
Epoch 69, Training loss 0.3555401563644409, Validation loss 0.33759650588035583
Epoch 70, Training loss 0.35551339387893677, Validation loss 0.3373274505138397
Epoch 71, Training loss 0.35545578598976135, Validation loss 0.3369501829147339
Epoch 72, Training loss 0.3554099202156067, Validation loss 0.3365986943244934
Epoch 73, Training loss 0.3554004728794098, Validation loss 0.3363669216632843
Epoch 74, Training loss 0.35541170835494995, Validation loss 0.3362954258918762
Epoch 75, Training loss 0.3554095923900604, Validation loss 0.33638501167297363
Epoch 76, Training loss 0.3553798794746399, Validation loss 0.3366064131259918
Epoch 77, Training loss 0.355339914560318, Validation loss 0.3368972837924957
Epoch 78, Training loss 0.35531502962112427, Validation loss 0.3371675908565521
Epoch 79, Training loss 0.35531070828437805, Validation loss 0.3373277187347412
Epoch 80, Training loss 0.35531044006347656, Validation loss 0.33732736110687256
Epoch 81, Training loss 0.3552969694137573, Validation loss 0.33717843890190125
Epoch 82, Training loss 0.3552703559398651, Validation loss 0.33694517612457275
Epoch 83, Training loss 0.355244904756546, Validation loss 0.3367106318473816
Epoch 84, Training loss 0.35523155331611633, Validation loss 0.33654287457466125
Epoch 85, Training loss 0.35522663593292236, Validation loss 0.3364788591861725
Epoch 86, Training loss 0.3552180826663971, Validation loss 0.3365233540534973
Epoch 87, Training loss 0.35520005226135254, Validation loss 0.3366541862487793
Epoch 88, Training loss 0.35517844557762146, Validation loss 0.33682578802108765
Epoch 89, Training loss 0.3551623821258545, Validation loss 0.3369802236557007
Epoch 90, Training loss 0.35515350103378296, Validation loss 0.3370654284954071
Epoch 91, Training loss 0.3551456034183502, Validation loss 0.3370566964149475

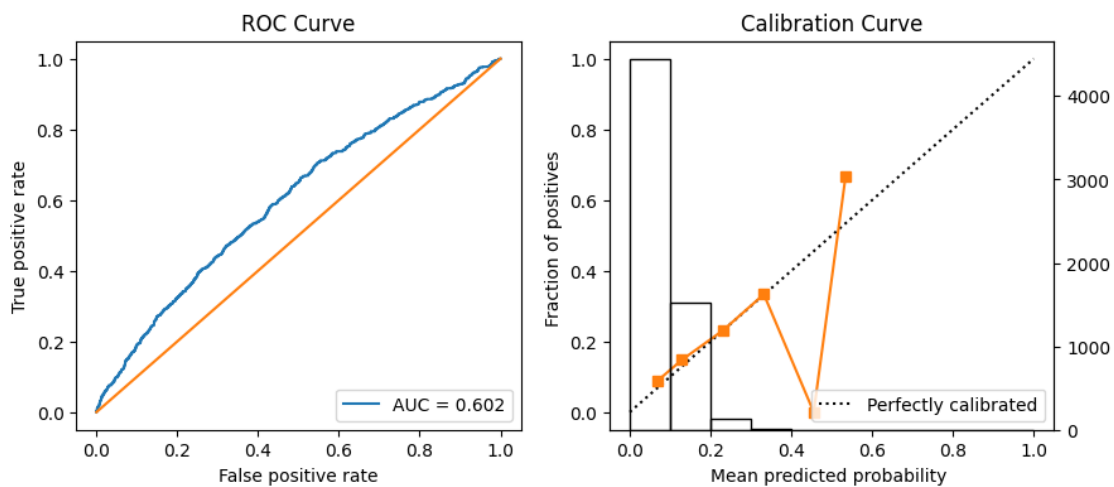
Epoch 92, Training loss 0.355133056640625, Validation loss 0.336965411901474
Epoch 93, Training loss 0.3551167845726013, Validation loss 0.33683180809020996
Epoch 94, Training loss 0.3551023602485657, Validation loss 0.3367052674293518
Epoch 95, Training loss 0.3550926744937897, Validation loss 0.3366256654262543
Epoch 96, Training loss 0.35508519411087036, Validation loss 0.33661308884620667
Epoch 97, Training loss 0.3550756573677063, Validation loss 0.336665540933609
Epoch 98, Training loss 0.3550633490085602, Validation loss 0.3367617726325989
Epoch 99, Training loss 0.35505130887031555, Validation loss 0.3368672728538513
Epoch 100, Training loss 0.3550421893596649, Validation loss 0.33694565296173096
Epoch 101, Training loss 0.35503512620925903, Validation loss 0.3369717597961426
Epoch 102, Training loss 0.35502752661705017, Validation loss 0.3369414508342743
Epoch 103, Training loss 0.3550183176994324, Validation loss 0.33687201142311096
Epoch 104, Training loss 0.3550090193748474, Validation loss 0.3367931842803955
Epoch 105, Training loss 0.35500144958496094, Validation loss 0.3367346227169037
Epoch 106, Training loss 0.3549954891204834, Validation loss 0.33671581745147705
Epoch 107, Training loss 0.3549894392490387, Validation loss 0.3367409408092499
Epoch 108, Training loss 0.35498252511024475, Validation loss
0.33679965138435364
Epoch 109, Training loss 0.35497549176216125, Validation loss 0.3368709087371826
Epoch 110, Training loss 0.35496950149536133, Validation loss
0.33693021535873413
Epoch 111, Training loss 0.35496464371681213, Validation loss 0.3369589149951935
Epoch 112, Training loss 0.3549599349498749, Validation loss 0.33695098757743835
Epoch 113, Training loss 0.3549547493457794, Validation loss 0.33691462874412537
Epoch 114, Training loss 0.3549495339393616, Validation loss 0.3368677496910095
Epoch 115, Training loss 0.35494503378868103, Validation loss 0.3368299901485443
Epoch 116, Training loss 0.3549412488937378, Validation loss 0.3368152678012848
Epoch 117, Training loss 0.3549376428127289, Validation loss 0.33682781457901
Epoch 118, Training loss 0.35493379831314087, Validation loss 0.3368615210056305
Epoch 119, Training loss 0.35492992401123047, Validation loss 0.336902916431427
Epoch 120, Training loss 0.35492655634880066, Validation loss
0.33693641424179077
Epoch 121, Training loss 0.35492363572120667, Validation loss 0.3369505703449249
Epoch 122, Training loss 0.3549208343029022, Validation loss 0.3369424343109131
Epoch 123, Training loss 0.35491788387298584, Validation loss 0.3369179964065552
Epoch 124, Training loss 0.35491499304771423, Validation loss 0.3368891477584839
Epoch 125, Training loss 0.3549124002456665, Validation loss 0.33686843514442444
Epoch 126, Training loss 0.35491010546684265, Validation loss 0.3368642330169678
Epoch 127, Training loss 0.3549078404903412, Validation loss 0.33687806129455566
Epoch 128, Training loss 0.35490548610687256, Validation loss 0.3369044065475464
Epoch 129, Training loss 0.3549031913280487, Validation loss 0.3369336724281311
Epoch 130, Training loss 0.35490110516548157, Validation loss 0.336955726146698
Epoch 131, Training loss 0.35489922761917114, Validation loss 0.3369642198085785
Epoch 132, Training loss 0.3548973500728607, Validation loss 0.33695876598358154
Epoch 133, Training loss 0.35489538311958313, Validation loss 0.3369443416595459
Epoch 134, Training loss 0.3548935055732727, Validation loss 0.3369293212890625
Epoch 135, Training loss 0.35489174723625183, Validation loss
0.33692100644111633

Epoch 136, Training loss 0.3548900783061981, Validation loss 0.3369234800338745
 Epoch 137, Training loss 0.354888379573822, Validation loss 0.33693578839302063
 Early stopping! Epoch 137
 Best model at epoch 37

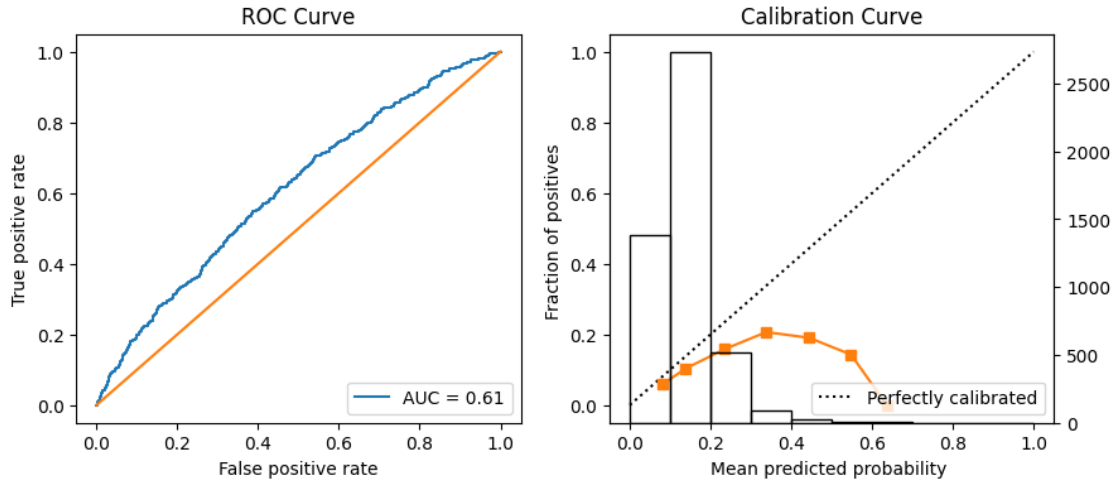
4 Evaluate the Partial Response Network

```
[21]: y_test_prn_pytorch = prn.predict(x_test_tensor, device=device).cpu().numpy()
      y_val_prn_pytorch = prn.predict(x_val_tensor, device=device).cpu().numpy()

      prn_metrics_test = modelStats(y_test_prn_pytorch, y_test, y_train, auc_ci=True)
      modelStats(y_val_prn_pytorch, y_val, y_train, auc_ci=True)
```



```
----- Metrics -----
prevalence: 0.123
sensitivity: 0.235
specificity: 0.867
accuracy: 0.799
ppv: 0.174
auc score: 0.602
auc lower ci: 0.578
auc upper ci: 0.622
-----
```



```

----- Metrics -----
prevalence: 0.123
sensitivity: 0.639
specificity: 0.512
accuracy: 0.524
ppv: 0.126
auc score: 0.61
auc lower ci: 0.583
auc upper ci: 0.632
-----

```

```

[21]: {'prevalence': 0.123,
      'sensitivity': 0.639,
      'specificity': 0.512,
      'accuracy': 0.524,
      'ppv': 0.126,
      'auc score': 0.61,
      'auc lower ci': '0.583',
      'auc upper ci': '0.632'}

```

5 LASSO on the Partial Response Network

```

[22]: # Check current device memory usage
      print_tensor_info()

```

```

Tensors currently in memory:
Name: x_train_tensor, Type: Tensor, Size: torch.Size([31315, 11]), Device:
cuda:0, Dtype: torch.float32
Name: x_test_tensor, Type: Tensor, Size: torch.Size([6120, 11]), Device: cuda:0,

```

Dtype: torch.float32
 Name: x_val_tensor, Type: Tensor, Size: torch.Size([4750, 11]), Device: cuda:0,
 Dtype: torch.float32
 Name: Unknown, Type: NestedTensor, Size: torch.Size([1, j0, 3]), Device: meta,
 Dtype: torch.float32
 Name: Unknown, Type: Tensor, Size: torch.Size([3, 3]), Device: meta, Dtype:
 torch.float32
 Name: Unknown, Type: Tensor, Size: torch.Size([2]), Device: meta, Dtype:
 torch.int64
 Name: Unknown, Type: Tensor, Size: torch.Size([10, 11]), Device: cuda:0, Dtype:
 torch.float32
 Name: Unknown, Type: Tensor, Size: torch.Size([10]), Device: cuda:0, Dtype:
 torch.float32
 Name: Unknown, Type: Tensor, Size: torch.Size([1, 10]), Device: cuda:0, Dtype:
 torch.float32
 Name: Unknown, Type: Tensor, Size: torch.Size([1]), Device: cuda:0, Dtype:
 torch.float32
 Name: Unknown, Type: Tensor, Size: torch.Size([6120, 1]), Device: cpu, Dtype:
 torch.float32
 Name: Unknown, Type: Tensor, Size: torch.Size([4750, 1]), Device: cpu, Dtype:
 torch.float32
 Name: partial_responses_train, Type: Tensor, Size: torch.Size([31315, 66]),
 Device: cpu, Dtype: torch.float32
 Name: partial_responses_test, Type: Tensor, Size: torch.Size([6120, 66]),
 Device: cpu, Dtype: torch.float32
 Name: Unknown, Type: Parameter, Size: torch.Size([10, 11]), Device: cuda:0,
 Dtype: torch.float32
 Name: Unknown, Type: Parameter, Size: torch.Size([10]), Device: cuda:0, Dtype:
 torch.float32
 Name: Unknown, Type: Parameter, Size: torch.Size([1, 10]), Device: cuda:0,
 Dtype: torch.float32
 Name: Unknown, Type: Parameter, Size: torch.Size([1]), Device: cuda:0, Dtype:
 torch.float32
 Name: Unknown, Type: Tensor, Size: torch.Size([15]), Device: cpu, Dtype:
 torch.float32
 Name: Unknown, Type: Tensor, Size: torch.Size([15]), Device: cpu, Dtype:
 torch.float32
 Name: Unknown, Type: Tensor, Size: torch.Size([15]), Device: cpu, Dtype:
 torch.float32
 Name: Unknown, Type: Tensor, Size: torch.Size([15]), Device: cpu, Dtype:
 torch.float32
 Name: Unknown, Type: Tensor, Size: torch.Size([2]), Device: cpu, Dtype:
 torch.float32
 Name: Unknown, Type: Tensor, Size: torch.Size([2]), Device: cpu, Dtype:
 torch.float32
 Name: Unknown, Type: Tensor, Size: torch.Size([2]), Device: cpu, Dtype:
 torch.float32
 Name: Unknown, Type: Tensor, Size: torch.Size([15]), Device: cpu, Dtype:

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

```

torch.float32
Name: Unknown, Type: Tensor, Size: torch.Size([30, 2]), Device: cpu, Dtype:
torch.float32
Name: Unknown, Type: Tensor, Size: torch.Size([4, 2]), Device: cpu, Dtype:
torch.float32
Name: Unknown, Type: Tensor, Size: torch.Size([4, 2]), Device: cpu, Dtype:
torch.float32
Name: Unknown, Type: Tensor, Size: torch.Size([4, 2]), Device: cpu, Dtype:
torch.float32
Name: Unknown, Type: Parameter, Size: torch.Size([1]), Device: cuda:0, Dtype:
torch.float32
Name: Unknown, Type: Parameter, Size: torch.Size([1, 55]), Device: cuda:0,
Dtype: torch.float32
Name: Unknown, Type: Parameter, Size: torch.Size([55]), Device: cuda:0, Dtype:
torch.float32
Name: Unknown, Type: Parameter, Size: torch.Size([55, 11]), Device: cuda:0,
Dtype: torch.float32
Name: Unknown, Type: Tensor, Size: torch.Size([55, 11]), Device: cuda:0, Dtype:
torch.float32
Name: Unknown, Type: Tensor, Size: torch.Size([55, 11]), Device: cuda:0, Dtype:
torch.float32
Name: Unknown, Type: Tensor, Size: torch.Size([55]), Device: cuda:0, Dtype:
torch.float32
Name: Unknown, Type: Tensor, Size: torch.Size([1, 55]), Device: cuda:0, Dtype:
torch.float32
Name: Unknown, Type: Tensor, Size: torch.Size([1]), Device: cuda:0, Dtype:
torch.float32
Name: Unknown, Type: Tensor, Size: torch.Size([6120, 1]), Device: cpu, Dtype:
torch.float32
Name: Unknown, Type: Tensor, Size: torch.Size([4750, 1]), Device: cpu, Dtype:
torch.float32

```

Device Memory Usage:

CUDA (GPU):

Allocated: 0.02 GB

Cached: 0.07 GB

c:\Users\localuser\PRiSM\prism_github\venv_prism\Lib\site-

packages\torch\distributed\distributed_c10d.py:366: UserWarning:

torch.distributed.reduce_op is deprecated, please use torch.distributed.ReduceOp
instead

warnings.warn(

```

[23]: # Lowering batch size from to 256 to accomodate larger PRN model compared to
      ↪ the MLP
      # by default, max_workers (threads) = logical CPU cores - 1. Otherwise, set
      ↪ optional argument max_workers
      # for single GPU (cuda): choose max_workers=1.

```

```

# for MPS (Apple Silicon): start with max_workers=1, then experiment with more
partial_responses_params_prn = {
    'x_train' : x_train_tensor,
    'method' : method,
    'device' : device,
    'batch_size' : 256,
    'max_workers' : 1
}

partial_responses_train_prn = partial_responses(x_train_tensor, prn,
↪**partial_responses_params_prn)

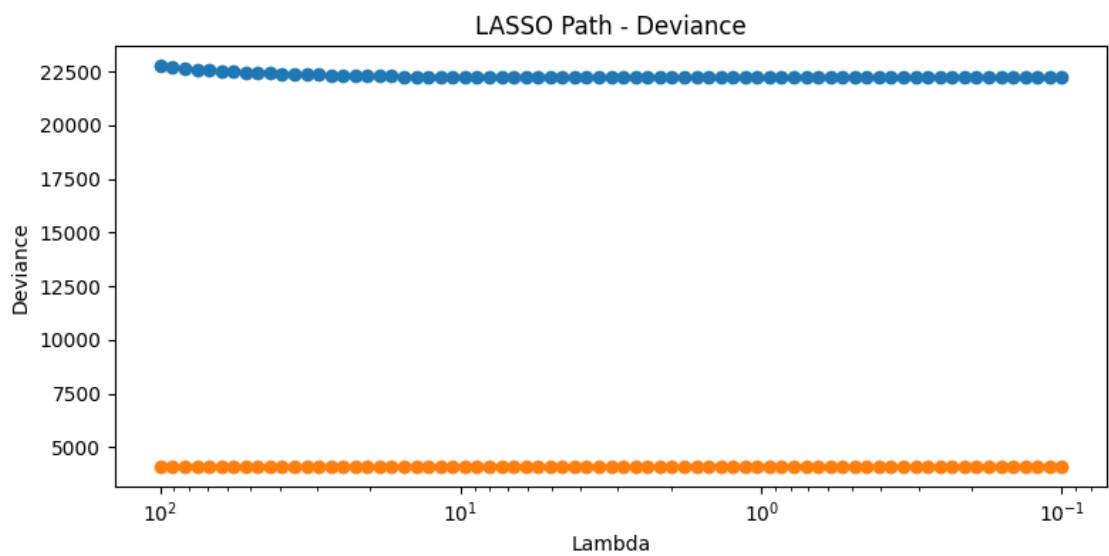
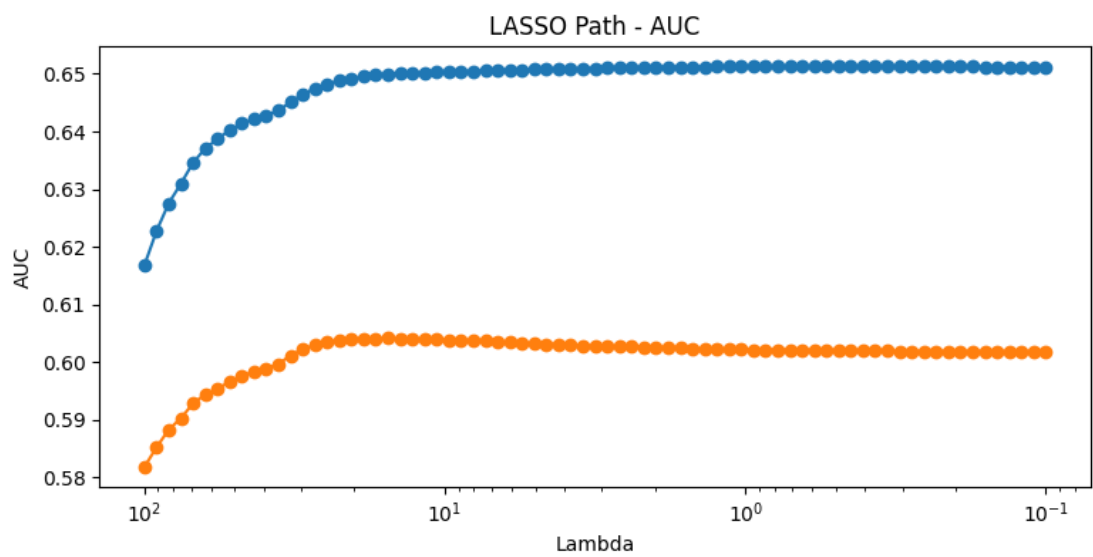
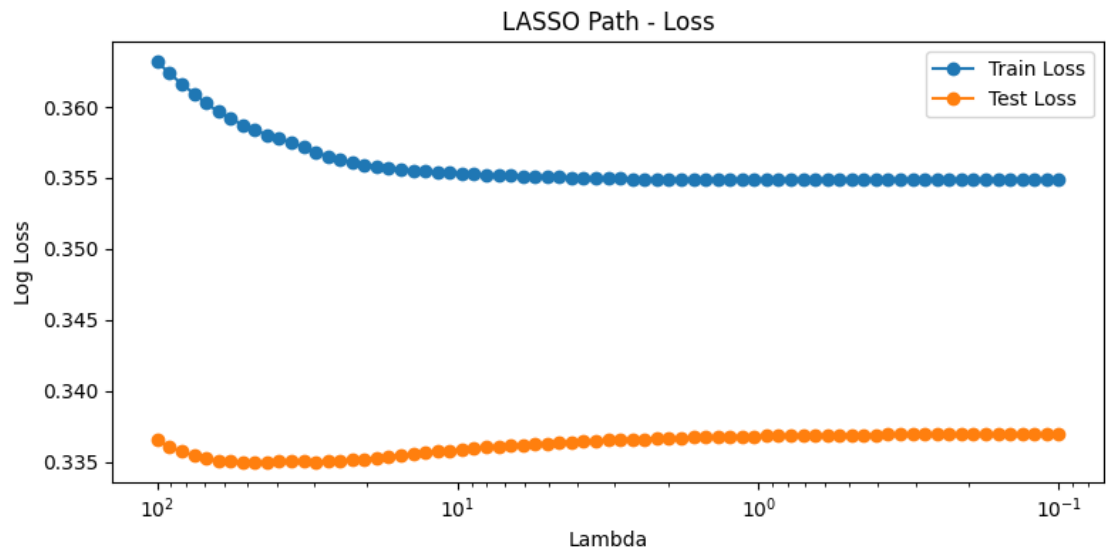
partial_responses_test_prn = partial_responses(x_test_tensor, prn,
↪**partial_responses_params_prn)

```

```

[24]: %reload_ext autoreload
lasso_results_prn = lasso(
    partial_responses_train_prn,
    partial_responses_test_prn,
    y_train,
    y_test,
    feature_names=feature_names,
    nlambdas=75,
    min_lambda=0.1,
    max_lambda=100,
    batch_size=2,
    seed=seed
)

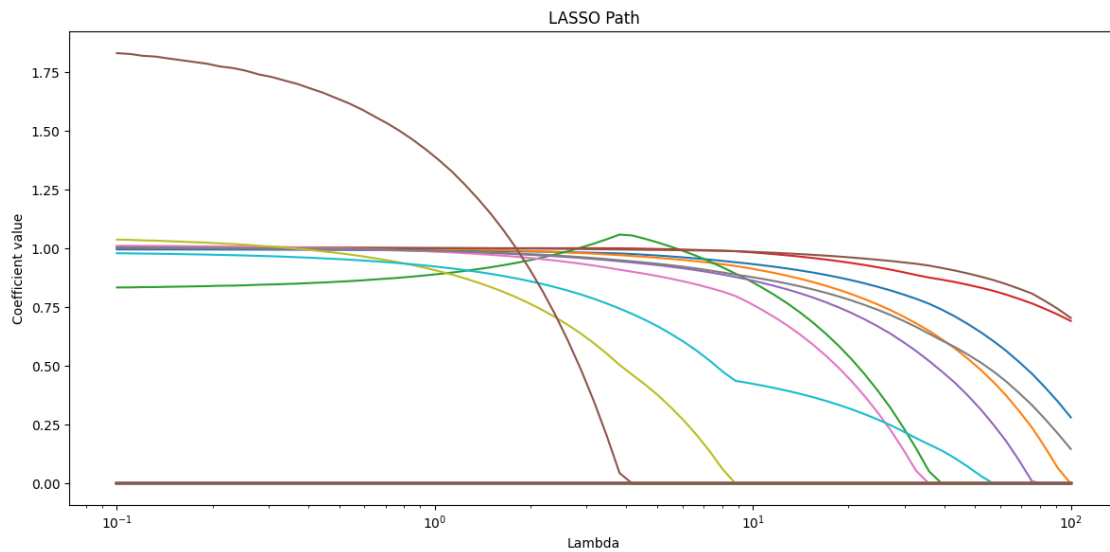
```



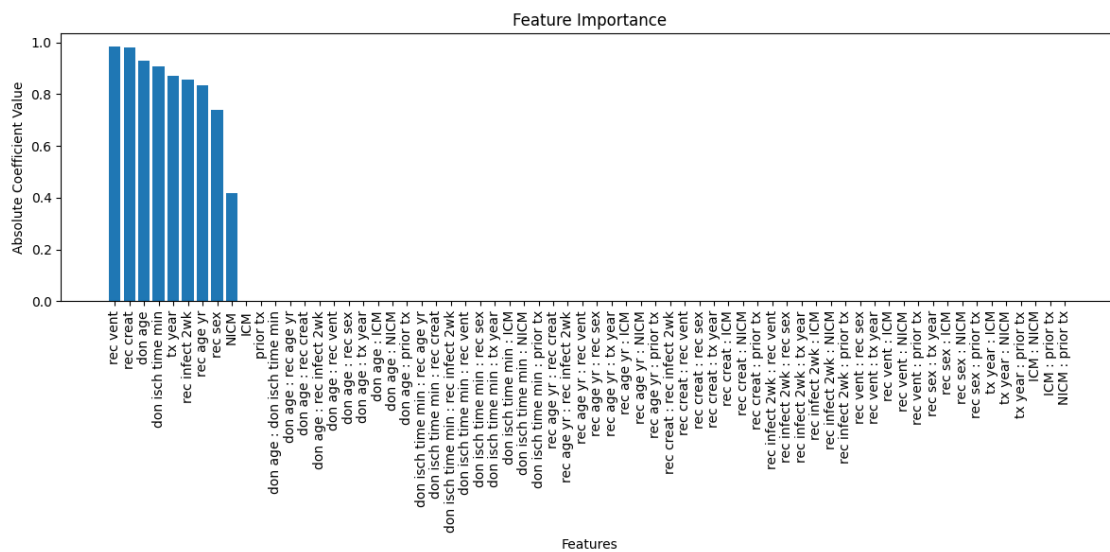
<IPython.core.display.HTML object>

All fits converged successfully.

```
[25]: lasso_results_prn.plot_lambda_path()
```



```
[26]: lasso_results_prn.select_lambda(24)
lasso_results_prn.plot_feature_importance()
```



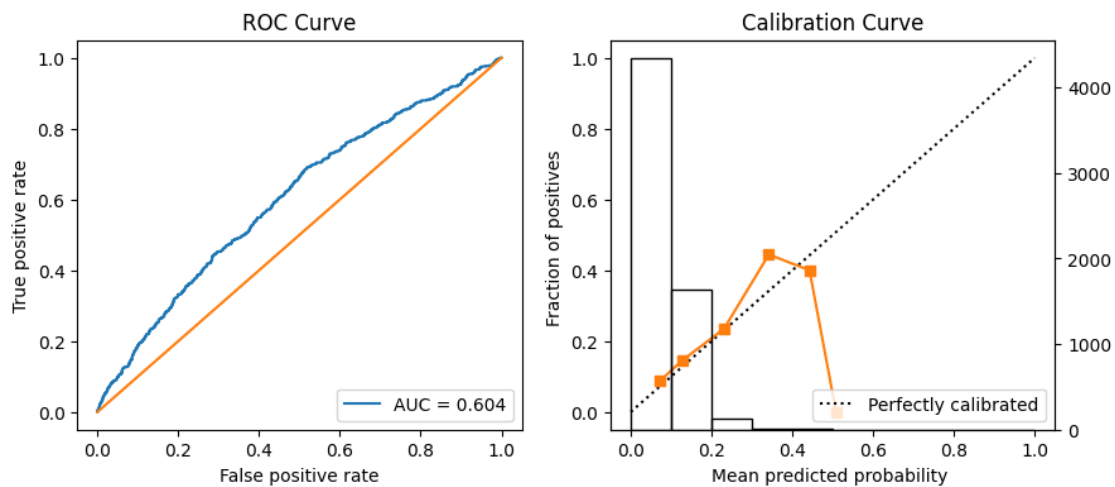
6 Validation inference

```
[27]: lasso_results_prn.select_lambda(24)
prn_lasso = lasso_results_prn.get_selected_model()
```

```
[28]: partial_responses_params_prn = {
    'x_train' : x_train_tensor,
    'method' : method,
    'device' : device,
    'batch_size' : 256,
    'max_workers' : 1
}

partial_responses_val_prn = partial_responses(x_val_tensor, prn,
    ↪**partial_responses_params_prn)
```

```
[29]: y_pred_test_prn_lasso = prn_lasso.predict_proba(partial_responses_test_prn.
    ↪cpu().numpy())[:, 1]
lasso_metrics_test = modelStats(y_pred_test_prn_lasso, y_test, y_train,
    ↪auc_ci=True)
```

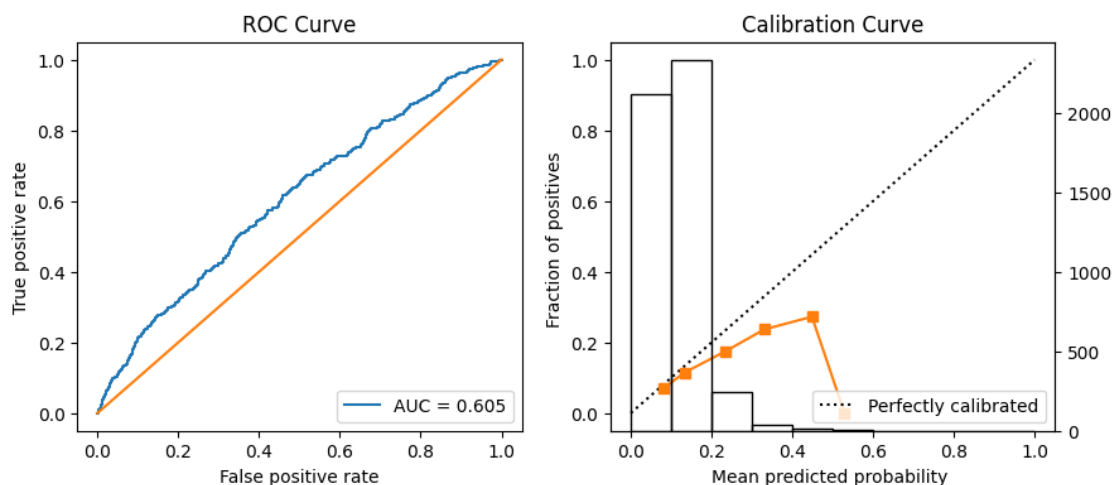


```
----- Metrics -----
prevalence: 0.123
sensitivity: 0.235
specificity: 0.867
accuracy: 0.799
ppv: 0.174
auc score: 0.604
auc lower ci: 0.579
auc upper ci: 0.625
```

```

[30]: y_pred_val_prn_lasso = prn_lasso.predict_proba(partial_responses_val_prn.cpu().
        ↪numpy())[:, 1]
modelStats(y_pred_val_prn_lasso, y_val, y_train, auc_ci=True)

```



```

----- Metrics -----
prevalence: 0.123
sensitivity: 0.456
specificity: 0.673
accuracy: 0.652
ppv: 0.133
auc score: 0.605
auc lower ci: 0.581
auc upper ci: 0.627
-----

```

```

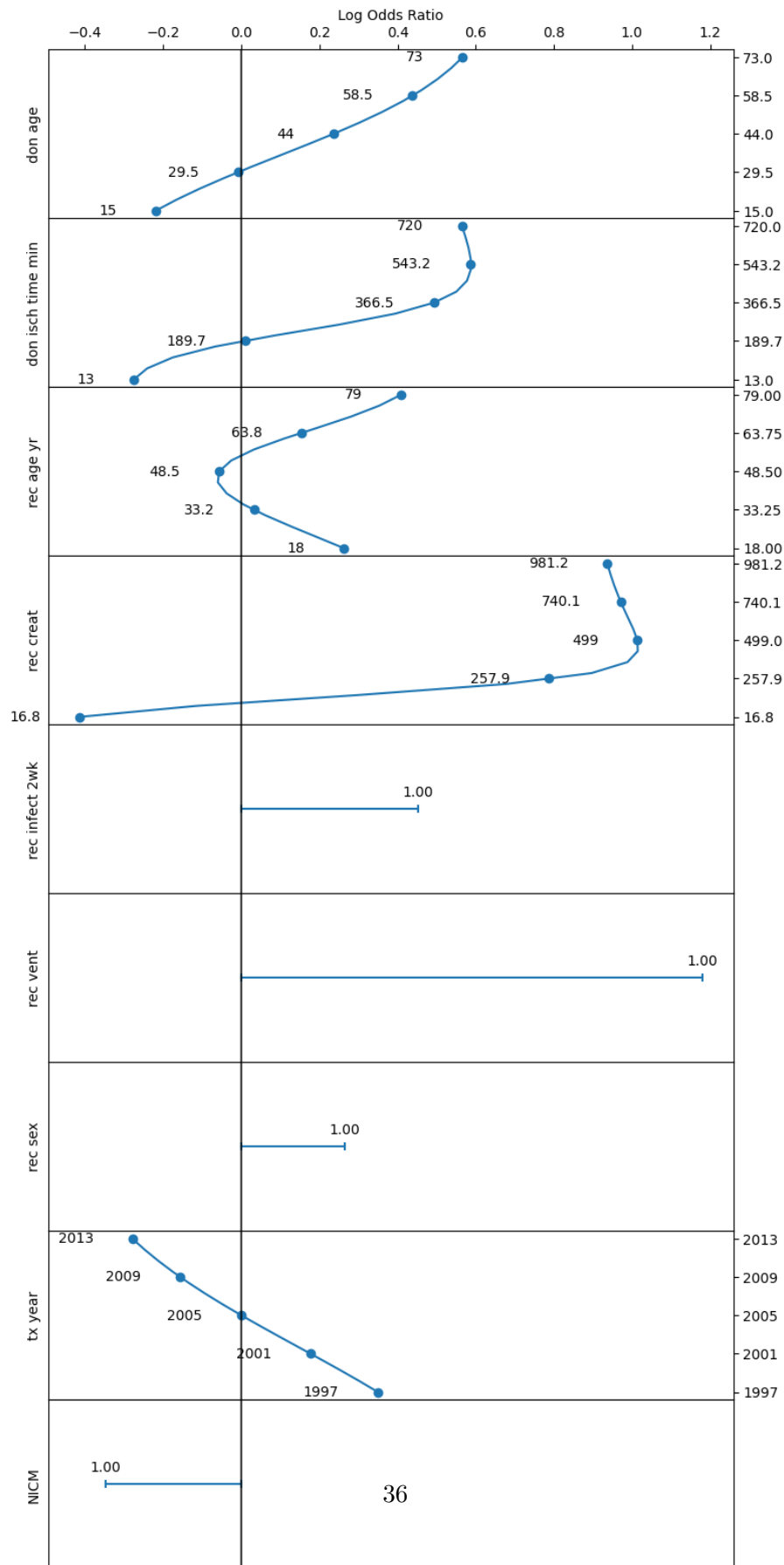
[30]: {'prevalence': 0.123,
      'sensitivity': 0.456,
      'specificity': 0.673,
      'accuracy': 0.652,
      'ppv': 0.133,
      'auc score': 0.605,
      'auc lower ci': '0.581',
      'auc upper ci': '0.627'}

```

7 Partial Response Network Nomogram

```
[31]: nomogram(  
    lasso_results_prn,  
    x_train_tensor,  
    x_train0.median().values,  
    x_train0.std().values,  
    prn,  
    n_steps=15,  
    sd_scale=2,  
    method=method,  
    device=device,  
    categorical_threshold=15,  
    subtract_univariate=True  
);
```

Nomogram of univariate and mixed bivariate partial responses



7.1 Nomogram with more

TODO: modify `lasso_results` to include possibility to select all features, all univariate, all bivariate (e.g. for plotting).

```
[32]: # test with more features  
# todo add all features method to lasso_results object.  
lasso_results_prn.select_lambda(74)  
nomogram(  
    lasso_results_prn,  
    x_train_tensor,  
    x_train0.median().values,  
    x_train0.std().values,  
    prn,  
    n_steps=15,  
    sd_scale=2,  
    method=method,  
    device=device,  
    categorical_threshold=15  
);
```

Nomogram of univariate and mixed bivariate partial responses

